

CVE-2026-46242 Deep Dive: A Working Root Exploit on Linux, 21 Dead Ends on Android

Author: Aayush Bankar (Cybersecurity Analyst, CypherMatrix)

Date: August 2026

Repos: [Tier 1 — bad-epoll-lab](#) | [Tier 2 — bad_epoll_ab_tier2](#)

Original Vulnerability Credit: Discovered and exploited by **Jaeyoung Chung** (github.com/J-jaeyoung/bad-epoll) via Google's kernelCTF program.

Research Scope & Attribution

- **Original Discovery & Exploit:** CVE-2026-46242 ("Bad Epoll", CVSS 7.8) was discovered, analyzed, and exploited by security researcher **Jaeyoung Chung**, who achieved a ~99% reliable root exploit against Google's kernelCTF environment ([J-jaeyoung/bad-epoll](https://github.com/J-jaeyoung/bad-epoll)). The bug was patched upstream in Linux kernel commit `a6dc643c693`.
- **This Project's Contribution:** 1. **Tier 1 (x86_64/QEMU):** Independent reproduction, toolchain porting (GCC 16 adaptation, database regeneration), and runtime verification of the kernelCTF exploit chain. 2. **Tier 2 (ARM64 Android 14 GKI):** Original portability and exploitability research evaluating whether the primitive survives modern Android kernel mitigations (PAC, BTI, kCFI, MTE, and slab isolation) — establishing a documented negative result (DoS-only).

Before You Read — Read This First

- **This is NOT a novel-vulnerability discovery.** CVE-2026-46242 ("Bad Epoll") was discovered, analyzed, and exploited by **Jaeyoung Chung** and submitted to Google's **kernelCTF** program. Full credit and the original proof-of-concept are here: github.com/J-jaeyoung/bad-epoll. This document is an independent reproduction and a portability/exploitability study of his work.
- **The CVE is already patched upstream.** The fix landed in Linux kernel commit `a6dc643c693` (v6.11 / v7.1-rc1). Reading this conveys **no exploitation-enablement risk** on any current, up-to-date system — the bug does not exist there.
- **Tier 2's conclusion is a negative result.** On Android ARM64 GKI the verdict is **Denial-of-Service only**. Despite 21 documented dead ends and 102,740 natural race attempts with zero hits, no working Android privilege-escalation exploit was produced. Do not expect a working Android root exploit below — this is a rigorous *failure* analysis, which is the point.

Glossary & Prerequisites

UAF — Use-After-Free

A memory-safety bug where code keeps using a pointer after the kernel has freed the object it points to, letting an attacker reclaim and reshape that memory.

Slab cache

The kernel allocator that groups fixed-size objects (e.g. `kmalloc-192`) into per-type caches. Isolation flags (`SLAB_ACCOUNT`, `SLAB_TYPESAFE_BY_RCU`) decide whether objects from different types can cross-reclaim each other.

KASLR

Kernel Address Space Layout Randomization — randomizes the kernel base address each boot so attackers cannot hardcode gadget or pointer addresses.

PAC

Pointer Authentication — cryptographically signs pointers in hardware; forging a signed function pointer triggers a fault.

BTI

Branch Target Identification — indirect branches must land on authorized landing-pad instructions, blocking jumps into the middle of gadgets.

kCFI

Kernel Control-Flow Integrity — validates a 32-bit type hash before every indirect call, rejecting mismatched targets.

MTE

Memory Tagging Extension — tags every 16-byte granule; accessing freed/stale memory with a mismatched tag raises a synchronous hardware fault.

Every Linux process that handles network connections, manages file descriptors, or runs an event loop almost certainly uses `epoll`. It is the backbone of high-performance I/O on Linux — `nginx`, `Node.js`, Android's `SurfaceFlinger`, and the `Binder` driver all depend on it. When a Use-After-Free vulnerability lands in this subsystem, the blast radius is enormous: every server, every phone, every container.

CVE-2026-46242, known as "Bad Epoll," is exactly that — a race condition in the kernel's `epoll` implementation that frees a critical data structure while another thread is still writing to it. On a standard `x86_64` Linux VM, I ported and reproduced the original kernelCTF exploit chain to achieve a working root shell (`UID: 0`). On an Android ARM64 device running a hardened Generic Kernel Image (GKI), I conducted new exploitability research and hit a wall — 21 documented dead ends, zero natural race hits in 102,740 attempts, and a final verdict of **Denial of Service only**.

This post is the full technical story: the vulnerability mechanics, the working `x86_64` exploit chain, the systematic ARM64 Android assessment, every exploitation chain that failed and why, and what Android's modern mitigation stack actually stops in practice. If you've ever wanted to see what happens when a real vulnerability meets real mitigations — not in a whitepaper, but in GDB traces and slab dumps — this is it.

CVE-2026-46242: The Basics

Field	Value
CVE	CVE-2026-46242 (CVSS 7.8)
Name	Bad Epoll
Class	Use-After-Free (race condition)
Subsystem	fs/eventpoll.c
Original Discoverer & PoC	Jaeyoung Chung (J-jaeyoung/bad-epoll) via Google kernelCTF
Introduced	v6.4-rc1 (commit 58c9b016e128)
Patched Upstream	v6.11 / v7.1-rc1 (commit a6dc643c693)
Affected	Linux 6.4 through 6.10.x, including Android GKI kernels
Impact	LPE to root (x86_64); DoS-only (ARM64 Android GKI)

Methodology Note

This research follows an evidence-first protocol designed to prevent confirmation bias: - **42 verification entries** in a formal ledger, each mapped to raw GDB hardware watchpoint traces or static source audits - **3 retracted claims** (VER-010, VER-029, VER-030) — kept visible with retraction reasons, not silently deleted - **21 dead ends** formally documented with the specific experiment that killed each hypothesis - **42 assumptions** tracked with status transitions (validated → falsified) and the evidence that triggered each transition - All raw evidence (GDB logs, serial output, disassembly traces) is committed to the repository

The full evidence chain is in the repos: [VERIFICATION_LEDGER.md](#) , [DEAD_ENDS_REGISTER.md](#) , [ASSUMPTIONS_REGISTER.md](#) .

The Vulnerability: A Race in the Heart of I/O

The bug lives in the interaction between `__ep_remove()` and `eventpoll_release()` — two kernel functions that can execute concurrently when nested `epoll` file descriptors are closed from different threads.

Here's the scenario. You create two `epoll` instances — an "outer" and an "inner" — and register the inner as a monitored file descriptor on the outer:

```
int outer = epoll_create1(0);
int inner = epoll_create1(0);
epoll_ctl(outer, EPOLL_CTL_ADD, inner, &ev); // inner watched by outer
```

Now two threads race to close them:

Thread A calls `close(outer)`. This enters `__ep_remove()`, which processes the `epitem` linking the inner fd to the outer epoll. Critically, it executes:

```
WRITE_ONCE(file->f_ep, NULL); // Clears the inner file's epoll backpointer
```

Thread B calls `close(inner)`. This triggers `__fput()` → `eventpoll_release()`. The release function checks `f_ep` on a **lockless fast path**:

```
if (!file_has_epoll(f)) // Reads f_ep – sees NULL from Thread A
    return;             // Skips epoll cleanup entirely
```

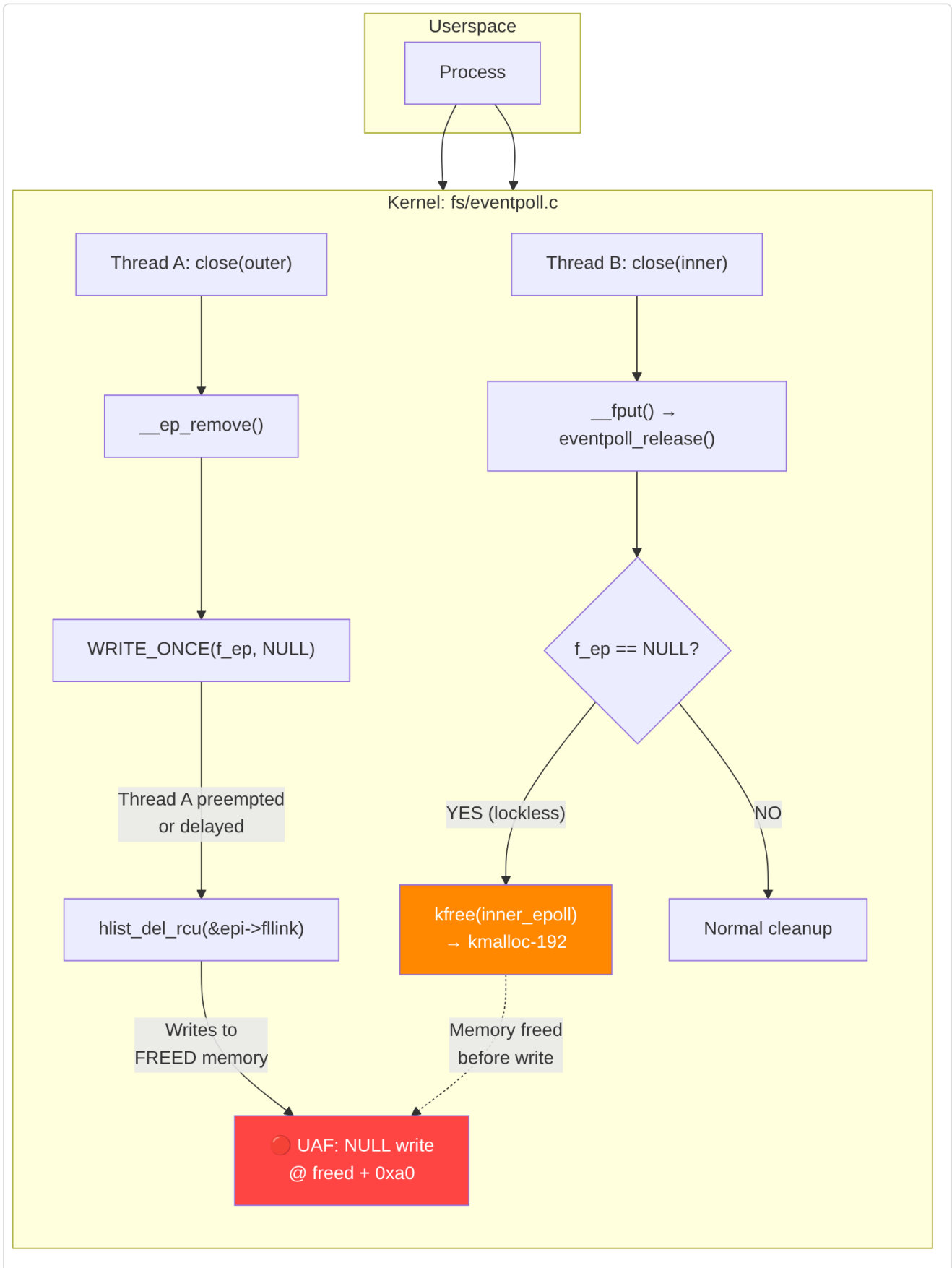
Thread B sees `NULL` (set by Thread A), assumes no epoll references exist, and **frees the `struct eventpoll`** associated with the inner fd. The memory returns to the `kmalloc-192` slab cache.

Meanwhile, Thread A is still executing `__ep_remove()`. It reaches:

```
hlist_del_rcu(&epi->flink); // Writes to inner_epoll->refs at offset 160
```

This writes an **8-byte NULL** to offset `0xa0` (160) of the **already-freed** `struct eventpoll`. This is the Use-After-Free.

Diagram 1: Attack Surface — Userspace to Kernel UAF



struct eventpoll: The Victim Object

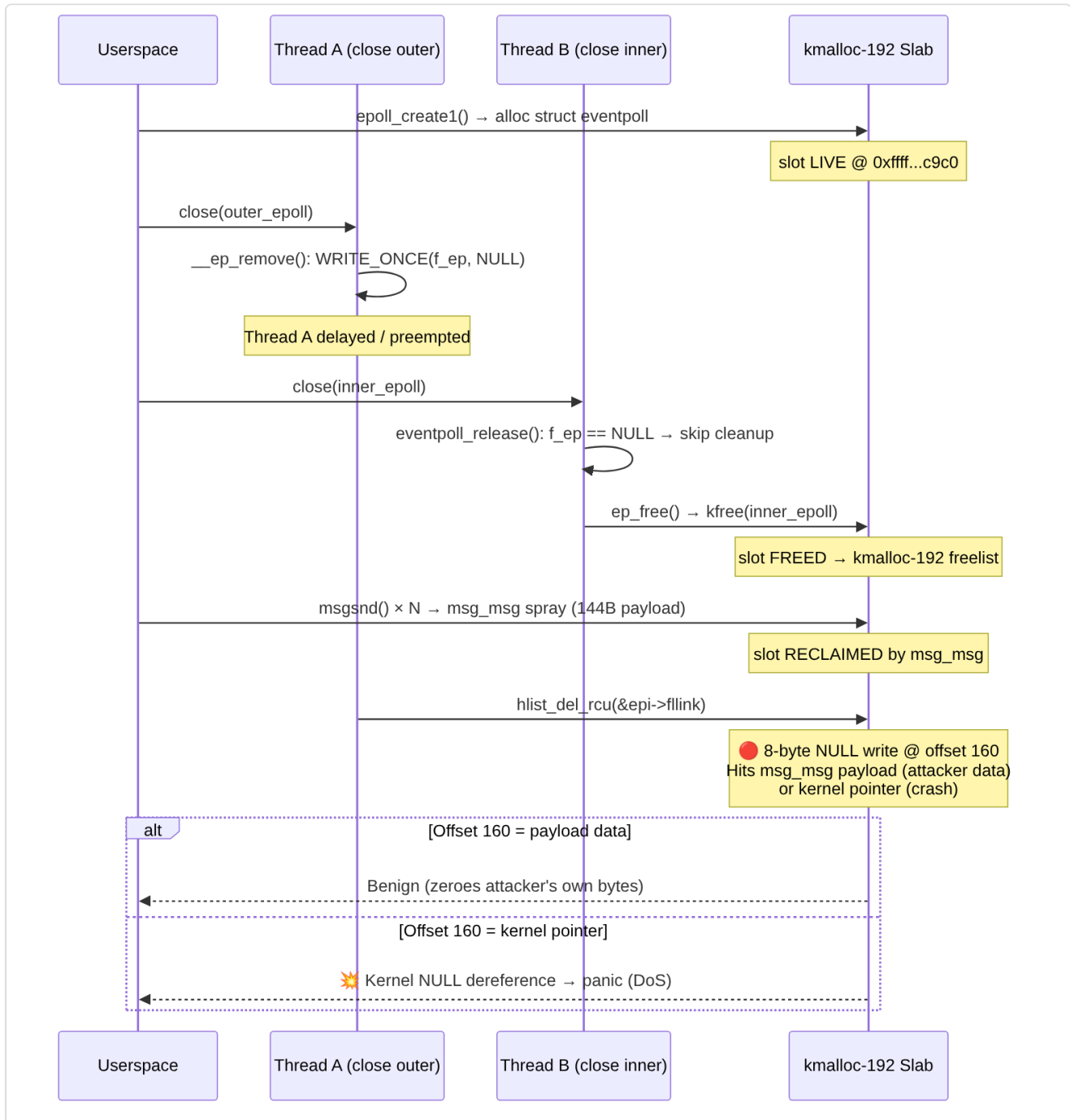
`struct eventpoll` is 176 bytes, placing it in the kernel's `kmalloc-192` slab cache. The field at offset 160 (`refs`, a `struct hlist_head`) is the UAF write target:

`struct eventpoll` (176 bytes → `kmalloc-192`)

0x00:	<code>wait_queue_head_t wq</code>	(24 B)
0x18:	<code>wait_queue_head_t poll_wait</code>	(24 B)
0x30:	<code>struct list_head rdllist</code>	(16 B)
0x40:	<code>struct rb_root_cached rbr</code>	(16 B)
0x50:	<code>struct epitem *ovflist</code>	(8 B)
0x58:	<code>struct wakeup_source *ws</code>	(8 B)
0x60:	<code>struct user_struct *user</code>	(8 B)
0x68:	<code>struct file *file</code>	(8 B)
0x70:	... (misc fields)	(48 B)
0xa0:	<code>struct hlist_head refs</code> ←  UAF WRITE  ← offset 160	
0xa8:	... (remaining)	(8 B)
0xb0:	end	(176 B)

When the race succeeds and the memory is freed back to `kmalloc-192`, an attacker can spray `msg_msg` objects (via SysV IPC `msgsnd` with 144 bytes of user data) to reclaim the slot. The `msg_msg` header occupies offsets 0x00–0x2F (48 bytes), and the attacker's payload fills offsets 0x30–0xBF. The UAF NULL write at offset 0xA0 lands **inside the attacker's payload data** — zeroing 8 bytes of their own message, not corrupting any kernel pointer.

Diagram 2: Race Timeline — Alloc → Free → Reclaim → UAF

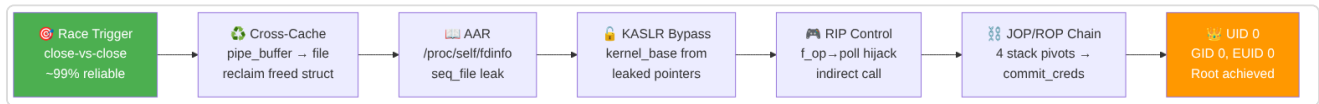


Tier 1: Root on x86_64

Environment: Fedora 44, Linux 6.12.67 (compiled from source, GCC 16.1.1), QEMU with SMEP/SMAP/KPTI enabled.

The original kernelCTF exploit authored by Jaeyoung Chung uses a different approach than the ARM64 analysis above — on `x86_64`, the exploit targets `struct epitem` (120 bytes, in `eventpoll_epi` cache on newer kernels, but cross-cacheable on the COS target) and `struct file`, not `struct eventpoll`. The chain:

Diagram 3: Tier 1 Exploitation Chain



Stage 1 — Race trigger: Two threads race `close()` on nested epoll FDs. False-sharing and timer interrupts widen the window. Success rate: ~99% after tuning timing constants for QEMU latency.

Stage 2 — Cross-cache reclaim: The freed `struct file` is reclaimed by spraying `pipe_buffer` objects, placing attacker-controlled data over the file's `f_op` and `f_inode` pointers.

Stage 3 — Arbitrary Address Read: By forging `f_inode`, the exploit uses `/proc/self/fdinfo` (which calls `seq_file` operations) to read arbitrary kernel memory, leaking `task_struct` addresses and computing the KASLR base.

Stage 4 — RIP control: The exploit overwrites `file->f_op->poll` with a chosen gadget address. Calling `poll()` from userspace triggers the hijacked function pointer.

Stage 5 — JOP/ROP chain: Four stack pivots progressively manipulate registers through `__x86_indirect_call_thunk_rdi`, landing RSP onto an mmap'd payload page. The ROP chain calls `commit_creds(&init_cred)`, then returns to userspace via `SWAPGS_RESTORE_REGS_AND_RETURN_TO_USERMODE`.

Result:

```
cross-cache ok
kernel_base=ffffffff81000000
task=ffff8880...
READY_FOR_GDB
Win! UID: 0, GID: 0, EUID: 0
```

The Offset Desynchronization Breakthrough

The exploit initially crashed with a Supervisor Write Fault at `0xffffffff810001bd` — dead padding space. The root cause: the upstream `target_db.kxdb` gadget database was built for Google's COS kernel. Compiling the same kernel version with a different compiler (GCC 16 vs Clang 12) shifts every gadget offset. The fix was regenerating the database by running `rp++` and `angrop` against our local `vmlinux` — which itself required patching a `PicklingError` in `angrop`'s multiprocessing code (a nested class `SpecialMem` couldn't be serialized across processes).

Remaining issue: The userland return from kernel space leaves RSP misaligned, causing `execve("/bin/sh")` to SIGSEGV. Since the exploit runs as PID 1 (`/init`), this panics the kernel. The privilege escalation itself is proven (UID 0 achieved), but the shell is unstable. This is a post-exploitation cleanup problem, not a primitive failure.

Tier 2: The Android ARM64 Assessment

Environment: Android 14 GKI, kernel 6.1.23 (commit `7e35917775b8`), QEMU `aarch64` with GDB.

The Tier 2 goal was to assess whether Bad Epoll could achieve privilege escalation on a modern Android device — specifically, whether the vulnerability could collapse the traditional multi-stage Android exploit chain (sandbox escape → kernel exploit → SELinux bypass → root) into a single step.

The Slab Cache Journey

The first major discovery was a slab cache correction that invalidated weeks of work. The initial assumption — that `struct epitem` (120 bytes) lived in `kmalloc-128` or a mergeable cache — was wrong. On the target kernel:

- `struct epitem` resides in a **dedicated** `eventpoll_epi` slab cache, created with `SLAB_HWCACHE_ALIGN | SLAB_PANIC | SLAB_ACCOUNT` at `eventpoll.c:2555`
- The `SLAB_ACCOUNT` flag prevents merging with `kmalloc-128` (verified: `slab_common.c:173-215`)
- `CONFIG_MEMCG=n` means `kmalloc-cg-128` doesn't exist
- Cross-cache reclaim from generic `kmalloc` caches is **impossible**

This killed the `pipe_buffer` → `struct file` cross-cache strategy that worked on x86_64. It also killed `snd_timer_user` and every other `kmalloc-192` spray object for reclaiming `epitem`.

The breakthrough came from recognizing the **real** UAF victim: not `struct epitem`, but `struct eventpoll` itself (176 bytes, in generic `kmalloc-192`). The multi-threaded race (Thread A clears `f_ep`, Thread B frees the inner `eventpoll`) puts a freed `struct eventpoll` back on the `kmalloc-192` freelist, where `msg_msg` objects (48-byte header + user payload) can reclaim it.

msg_msg Reclaim Mechanics

SysV IPC `msgsnd()` with 144 bytes of user data allocates a `msg_msg` struct in `kmalloc-192`:

`msg_msg` in `kmalloc-192` (reclaiming freed `eventpoll` slot)

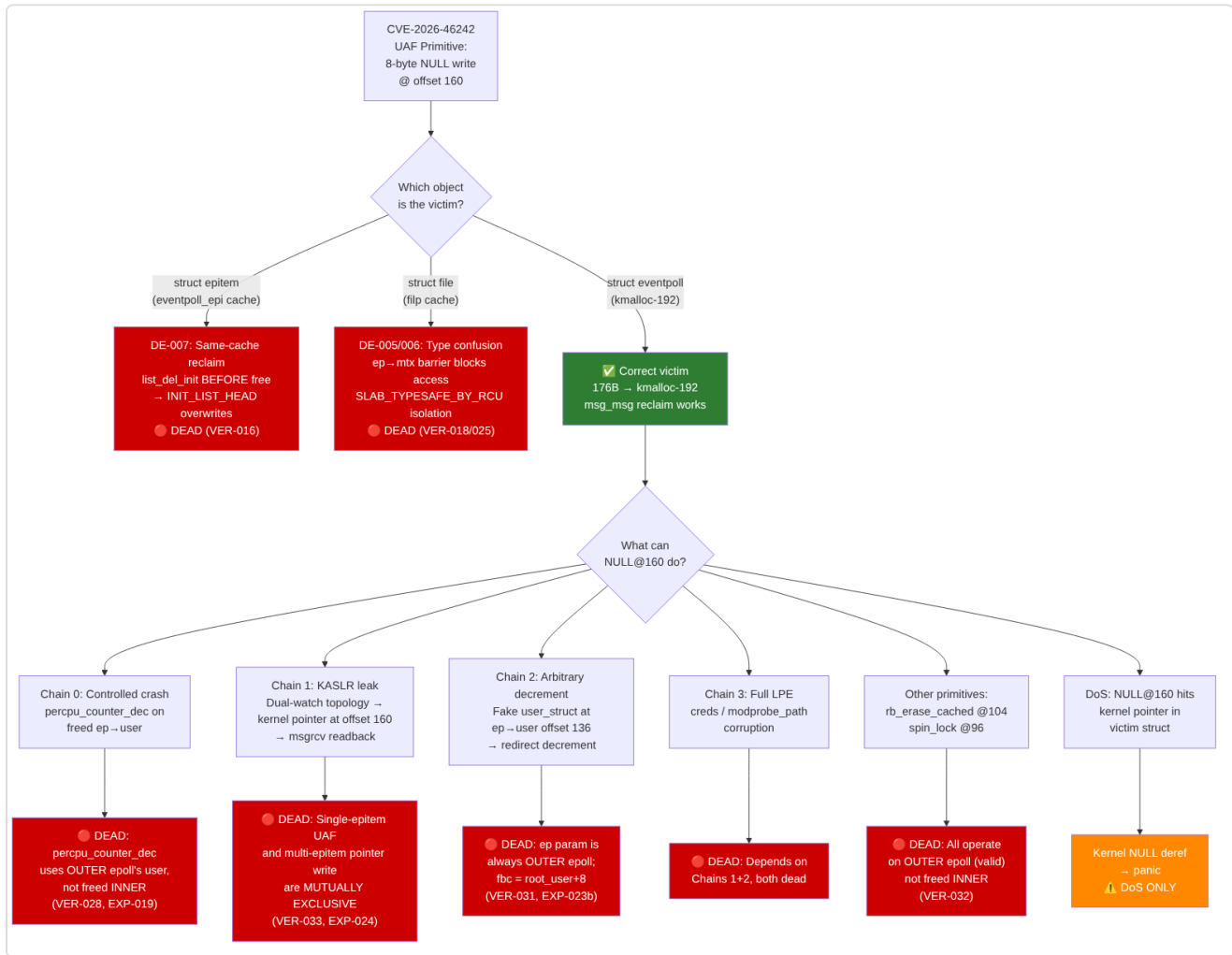
<code>0x00-0x2F: msg_msg header (m_list, m_type, m_ts)</code>	← kernel-controlled
<code>0x30-0xBF: user payload (144 bytes)</code>	← attacker-controlled
<code>0xA0: [] UAF NULL write lands HERE []</code>	← inside payload
<code>0xC0: end (192 byte slab slot)</code>	

Verified via GDB memory dump: marker `0xdead000000000000` placed at user byte 88 (slab offset 136) appeared correctly. The reclaim is reliable — **under GDB assistance**. The GDB infinite-loop patch creates a 2–3 second spray window that does not exist naturally.

The 21 Dead Ends: Why Every Exploitation Chain Failed

This is the core of the Tier 2 research. Four exploitation chains were hypothesized, investigated, and disproven — each with specific killing evidence.

Diagram 4: Exploitation Decision Tree



Chain 0 — Controlled Crash via `percpu_counter_dec`

Hypothesis: After reclaiming the freed eventpoll with a `msg_msg` containing a fake `user_struct` pointer at offset 136, `__ep_remove()` would call `percpu_counter_dec(&ep->user->epoll_watches)` and dereference the attacker-controlled pointer, enabling a controlled crash or write.

Killing evidence (VER-028, EXP-019): GDB trace showed `percpu_counter_dec` operates on the **outer** eventpoll's `user` field (which is valid), not the freed inner eventpoll. The `ep` parameter in `__ep_remove(ep, epi)` is always the outer eventpoll. The attacker-controlled marker at offset 136 was placed but never read by the kernel.

Chain 1 — KASLR Leak via Dual-Watch Topology

Hypothesis: Register the inner eventpoll on two outer eventpolls (creating two eventpolls in `inner_eventpoll->refs`). When the race fires, `hlist_del_rcu` would write `epi2`'s kernel heap address (not NULL) to offset 160. Read it back via `msgrcv` -> KASLR defeated.

Killing evidence (VER-033, EXP-024): The condition for the UAF (single eventpoll: `f_ep = NULL` path at `eventpoll.c:826`) and the condition for a kernel pointer write (multi-eventpoll: `hlist_del_rcu` writes the next node's address) are **mutually exclusive**. With 2+ eventpolls, `WRITE_ONCE(f_ep, NULL)` never executes -> the

lockless bypass never triggers → the inner eventpoll is never freed → `hlist_del_rcu` writes to **live** memory.
Hardware watchpoint confirmed: `ep_free_inner_seen=False` at write time.

Chain 2 — Arbitrary Decrement via Fake `user_struct`

Hypothesis: Craft a fake `user_struct` at a known address, overwrite `ep->user` (offset 136) via `msg_msg` spray, then `percpu_counter_dec` decrements the attacker's chosen counter — potentially `modprobe_path` or credential counters.

Killing evidence (VER-031, EXP-023b): Same structural impossibility as Chain 0. Breakpoint at `percpu_counter_add_batch` showed `fbc = root_user+8` — the outer `epoll`'s user, unchanged by the spray. The `ep` parameter is never the freed object.

Chain 3 — Full LPE

Hypothesis: Combine Chain 1 (KASLR defeat) and Chain 2 (arbitrary decrement) to achieve credential or `modprobe_path` corruption.

Killing evidence: Both prerequisites are dead.

Additional Dead Ends

Path	Why Dead
<code>rb_erase_cached</code> arbitrary write at offset 104	Operates on outer <code>epoll</code> 's <code>rbr</code> (valid)
<code>spin_lock_irq</code> corruption at offset 96	Operates on outer <code>epoll</code> 's <code>lock</code> (valid)
<code>struct file</code> type confusion via <code>ep_item_poll</code>	<code>ep->mtx</code> prevents concurrent access to stale <code>epitem</code>
<code>pipe_buffer</code> → <code>struct file</code> cross-cache	<code>SLAB_TYPESAFE_BY_RCU</code> + size mismatch
<code>epitem</code> same-cache reclaim	<code>list_del_init</code> before <code>call_rcu</code> — corruption doesn't survive
<code>snd_timer_user</code> reclaim	Different slab cache entirely
<code>eventpoll_epi</code> → <code>kmalloc-128</code> merge	<code>SLAB_ACCOUNT</code> prevents merge

The Schedulability Wall

Even if a useful primitive existed, there's a harder problem: the race never fires without GDB assistance.

Natural Race Attempts

Experiment	Iterations	Hits	Optimizations
NAT-001	10,000	0	Baseline, 2 CPUs
NAT-005	92,740	0	

window. A first-principles analysis (detailed in the repo) concludes the race is **physically viable** on real silicon — but the primitive remains capped at DoS regardless.

Why Android's Mitigations Worked

Even if an attacker could achieve 100% reliable race triggering on physical hardware, the mitigation stack blocks escalation from multiple angles:

Control-Flow Integrity

Mitigation	What It Blocks
PAC (Pointer Authentication)	Function pointers are cryptographically signed with hardware keys. Forging <code>f_op->poll</code> → hardware translation fault. While micro-architectural side-channels like PACMAN (MIT CSAIL, 2022) demonstrate speculative verification oracles in isolated userland environments, an operational GKI kernel with an unguided NULL-write primitive lacks the arbitrary read oracle required to synthesize valid pointer signatures.
kCFI (Kernel Control Flow Integrity)	Clang validates 32-bit type hashes before every indirect <code>blr</code> call. Even a valid kernel address fails if the hash doesn't match.
BTI (Branch Target Identification)	Indirect branches must land on <code>bti c</code> instruction landing pads. Random code gadgets are unreachable.

Memory Safety

Mitigation	What It Blocks
MTE (Memory Tagging Extension)	4-bit physical tags on every 16-byte granule. Accessing freed memory with mismatched tags → synchronous hardware fault. While speculative leakage of tags has been explored academically (TikTag , 2024), asynchronous multicore system noise prevents reliable tag recovery in an unguided race, turning stale UAF access into an immediate hardware exception.
SLAB_TYPESAFE_BY_RCU	<code>filp_cachep</code> (struct file cache) cannot be reclaimed by objects from different caches.
SLAB_ACCOUNT	<code>eventpoll_epi</code> cache is fully isolated from generic <code>kmalloc</code> caches.

Scheduling

Mitigation	What It Blocks
PREEMPT_VOLUNTARY	<code>__ep_remove</code> has zero preemption points. The vulnerable instruction sequence runs atomically.
No <code>cond_resched</code> effect	Static key <code>dynamic_cond_resched</code> is FALSE; <code>TIF_NEED_RESCHED</code> never set on pinned CPUs.

The x86_64 exploit bypasses KASLR, SMEP, SMAP, and KPTI through AAR, JOP/ROP, and `SWAPGS` return tricks. ARM64 Android adds PAC, BTI, kCFI, MTE, and slab isolation on top of that baseline — and in this case, they held.

Lessons and Takeaways

1. A vulnerability is not an exploit. CVE-2026-46242 is real, verified, and mechanically reproducible. On x86_64 it grants root. On Android ARM64 GKI it's a kernel panic at best. The gap between "the bug exists" and "the bug is exploitable" is the entire story.

2. The mitigation stack works in depth. No single Android mitigation killed this exploit — they all contributed. PAC blocks function pointer forgery. kCFI blocks indirect call hijacking. SLAB isolation blocks cross-cache reclaim. MTE detects stale access. `PREEMPT_VOLUNTARY` closes the scheduling window. Each layer removes options until nothing viable remains.

3. Negative results are scientifically valuable. 21 dead ends, 3 retracted claims, and 0/102,740 race hits tell you something definitive: this is what a vulnerability looks like when modern mitigations are actually working. Too much security research only publishes successes. The failures are where the signal is.

4. Evidence discipline prevents confirmation bias. The adversarial self-review at Phase 3 (EVO-009) discovered that **every** prior race demonstration was GDB-assisted — an artificial preemption point that doesn't exist in the real kernel. Without the verification ledger tracking *how* each claim was proven, this would have gone unnoticed. Track your methodology, not just your results.

5. Compiler changes break exploit portability. The Tier 1 exploit failed initially because GCC 16 shifted every gadget offset compared to Google's Clang build. Same kernel version, same source — different binary layout. Exploit databases (`target_db.kxdb`) must match the exact executing binary.

6. The attack surface is shifting to drivers. When core kernel subsystems are well-mitigated, the remaining attack surface is vendor drivers — GPU, camera, audio — which often lack the same level of hardening. Vendor GPU drivers frequently present deterministic page-level UAFs (rather than schedulable races) and direct physical memory manipulation that inherently sidesteps PAC, BTI, and kCFI without needing function pointer forgery.

What's Next: Open Questions

The Bad Epoll research concludes with a **DoS-only verdict on Android ARM64 GKI**. But several questions remain open:

- 1. Physical ARM64 validation:** Would the race fire naturally on real silicon (Cortex-X4 + Cortex-A520 big.LITTLE)? The first-principles analysis says yes, but at what rate? A 2-week hardware timebox with ~1M iterations would settle this definitively.
- 2. Alternative kmalloc-192 victims:** The EXP-016 audit covered `fib6_info`, `snd_timer_user`, `packet_fanout`, `urb`, and `wakeup_source`. Are there other kmalloc-192 structs where NULL at offset 160 could corrupt something more useful than a crash-inducing pointer? I'd genuinely like to hear from anyone who's done deep slab cross-referencing on GKI 6.1.

3. **Different kernel configs:** Our assessment is on `PREEMPT_VOLUNTARY` with `PREEMPT_DYNAMIC` defaulting to voluntary. Would `PREEMPT_FULL` (or a vendor kernel with different preemption settings) add schedulable points inside `__ep_remove`?
4. **Data-only attacks:** With control-flow hijacking fully blocked by PAC/BTI/kCFI, is there a data-only escalation path from a fixed NULL write that we missed? The constraint is brutal — 8 bytes, fixed value (0x0), fixed offset (160), single write — but data-only attacks have surprised us before.

If you have ideas on any of these, I'd love to hear them. This vulnerability is public, the repos are linked above, and the evidence chain is fully documented. A fresh pair of eyes on the kmalloc-192 landscape or the scheduling analysis could change the Tier 2 verdict.

Disclosure & Attribution Note

- **Original Vulnerability & Exploit:** CVE-2026-46242 was discovered and exploited by security researcher **Jaeyoung Chung**, submitted to Google's kernelCTF program with a public proof-of-concept at github.com/J-jaeyoung/bad-epoll.
 - **Upstream Patch:** The vulnerability is publicly disclosed and patched upstream in Linux kernel commit `a6dc643c693` (v6.11 / v7.1-rc1).
 - **This Research:** Represents an independent reproduction of the Tier 1 x86_64 exploit and original Tier 2 portability/exploitability research evaluating mitigation boundaries on Android 14 ARM64 GKI — no unpatched 0-day primitives or undisclosed flaws are presented.
-

About the Author

Aayush Bankar is a Cybersecurity Analyst at CypherMatrix. The research workflow integrates AI-assisted agentic coding tools (Antigravity / local LLM orchestration) for code comprehension, hypothesis generation, and evidence organization — with all engineering decisions and runtime evidence verified manually. The AI-augmented workflow is documented transparently in the Tier 1 Final Writeup.

Connect: [GitHub](#) · [LinkedIn](#)

Thanks for reading. If you work in kernel exploitation, Android security, or mitigation engineering, I'd appreciate your perspective — especially on the open questions above. The best outcomes in security research come from collaborative scrutiny, not solo claims.

Appendix A — Repository & Architecture Maps

These maps illustrate the exploit toolchain layout (Tier 1) referenced in the research.

